

Exercise 7: Financial Forecasting

1. Understanding Recursive Algorithms

Recursion is a programming technique where a method calls itself to solve a smaller version of the same problem until a base condition is reached.

A recursive solution typically consists of:

Base Case – Stops the recursion.

Recursive Case – Calls the same method with a smaller input.

Example- Factorial of 5:

$$5! = 5 \times 4!$$

$$4! = 4 \times 3!$$

$$3! = 3 \times 2!$$

$$2! = 2 \times 1!$$

$$1! = 1$$

Recursion simplifies problems that can be broken into smaller, similar subproblems.

FinancialForecast.java:

```
public class FinancialForecast {  
  
    public static double predictFutureValue(  
        double currentValue,  
        double growthRate,  
        int years) {  
  
        // Base Case  
        if (years == 0) {  
            return currentValue;  
        }  
    }  
}
```

```
// Recursive Case

return predictFutureValue(
    currentValue * (1 + growthRate),
    growthRate,
    years - 1);
}

public static void main(String[] args) {

    double currentValue = 10000;
    double growthRate = 0.10; // 10%
    int years = 5;

    double futureValue =
        predictFutureValue(
            currentValue,
            growthRate,
            years);

    System.out.printf(
        "Predicted value after %d years = ₹%.2f",
        years,
        futureValue);
}
}
```

Output:

```
● PS D:\SD\Digital-Nurture-JavaFSE-main\local repo\
e Adoptium\jdk-21.0.6.7-hotspot\bin\java.exe' '-X
k 1\Data Structure & Algorithms\Exercise-7\Financ
Predicted value after 5 years = Rs.16105.10
```

Analysis:

Time Complexity:

Let **n** be the number of years.

For the recursive solution:

predictFutureValue(value, rate, n)

Each recursive call reduces years by 1.

$$T(n) = T(n-1) + O(1)$$

Therefore:

$$\text{Time Complexity} = O(n)$$

Space Complexity:

Because each recursive call remains on the call stack:

$$\text{Space Complexity} = O(n)$$

Optimization Techniques:

Use Mathematical Formula:

Instead of repeated calculations:

```
double futureValue = currentValue * Math.pow(1 + growthRate, years);
```

Complexities:

$$\text{Time} \approx O(\log n)$$

$$\text{Space} = O(1)$$